

DSA STUDY BLUEPRINT SERIES

29. Search 2D Matrix Variation I and II

Search in Fully Sorted and Row/Col Sorted Matrices

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Type I:** Sorted as a single flat list. Perform flat 1D binary search: `arr[mid/cols][mid%cols]`.
- **Type II:** Sorted independently by row & col. Search from top-right corner.

Memory & Execution Blueprint

Matrix II Corner Search (Target = 12):

Start at $[0][col-1] = 15 \cdot 15 > 12 \Rightarrow$ Move Left • Next is
 $11 \cdot 11 < 12 \Rightarrow$ Move Down

C++ Implementation Reference

Standard code layout and syntax

```
bool searchMatrixII(vector<vector<int>>& mat, int target) {  
    int r = 0, c = mat[0].size() - 1;  
    while (r < mat.size() && c >= 0) {  
        if (mat[r][c] == target) return true;  
        if (mat[r][c] > target) c--;  
        else r++;  
    }  
    return false;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Search Matrix I	$O(\log(R * C))$
Search Matrix II	$O(R + C)$

Key Practices & Gotchas

- **Important:** Matrix II can also start search from bottom-left corner.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Row-Major index resolution mapping for element at 2D coordinate `[1][2]` (3x3 grid):

Row Index (i)	Col Index (j)	Formula: <code>index = i * cols + j</code>	Resolved 1D Offset
1	2	<code>index = 1 * 3 + 2</code>	Offset = 5 in linear memory block

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Row-Major Address Index mapping:** Flat memory locality makes sequential row scan speeds far faster than column-major scans.
- **Spiral Boundaries:** Keep strict checks inside individual boundary loop passes to avoid double print coordinates.
- **Related LeetCode Problems:** #54 Spiral Matrix, #74 Search a 2D Matrix, #48 Rotate Image.